

3.4.3 Operators and Operands (Arithmetic, Relational, Bitwise)

آپریٹنڈ (Operand) وہ ویلیو ہے جس پر عمل ہوتا ہے اور آپریٹر (Operator) وہ علامت ہے جو عمل کرتی ہے۔

CS XI

Chapter 3: Programming Fundamentals

Lecture 07 of 13

★ Golden

3.4.3 Operators and Operands (Arithmetic, Relational, Bitwise) ★ Golden

LEARN IT

An **operand** is the value or variable being worked on. An **operator** is the symbol that performs the action. Together they form an **expression**, which Python evaluates to a single value.

In `10 + 20`: **10** is an operand, **+** is the operator, **20** is an operand, and the result is **30**.

1. Arithmetic operators do maths:

Operator	Purpose	Example	Result
+	Addition	<code>5 + 3</code>	8
-	Subtraction	<code>10 - 4</code>	6
*	Multiplication	<code>4 * 2</code>	8
/	Division (result is always a float)	<code>9 / 2</code>	4.5
//	Floor division (drops the decimal part)	<code>9 // 2</code>	4
%	Modulus (remainder)	<code>9 % 2</code>	1
**	Exponentiation (power)	<code>2 ** 3</code>	8

```
print(5 + 3)
print(10 - 4)
print(4 * 2)
print(9 / 2)
print(9 // 2)
print(9 % 2)
print(2 ** 3)
print(8 / 2)
```

OUTPUT

```
8
6
8
4.5
4
1
8
4.0
```

Notice the last line: `8 / 2` gives `4.0`, not `4`, because `/` always gives a float.

Order of operations (precedence). Python follows the same order as maths (like BODMAS): brackets first, then `**`, then `*` `/` `//` `%` (left to right), then `+` `-` (left to right). `**` is the exception: it works right to left.

```
print(2 + 3 * 4)
print((2 + 3) * 4)
print(2 ** 3 ** 2)
print(10 - 4 - 3)
print(-9 // 2)
print(17 % 5, 17 // 5)
```

OUTPUT

```
14
20
512
3
-5
2 3
```

`2 ** 3 ** 2 = 2 ** 9 = 512` (right to left). `-9 // 2` is `-5` because floor division always rounds **down** to the smaller whole number (`-4.5 → -5`).

2. Assignment operators store a value in a variable, often updating it. The short forms are called **compound assignment**.

Operator	Example	Equivalent to
<code>=</code>	<code>x = 5</code>	Assign 5 to x
<code>+=</code>	<code>x += 2</code>	<code>x = x + 2</code>
<code>-=</code>	<code>x -= 1</code>	<code>x = x - 1</code>
<code>*=</code>	<code>x *= 3</code>	<code>x = x * 3</code>
<code>/=</code> , <code>/=</code> , <code>%=</code> , <code>**=</code>	<code>x /= 2</code>	<code>x = x / 2</code> (and so on)

WORKED EXAMPLE

Tracing assignment operators. Follow the value of `x` line by line:

```
x = 5
x += 2    # x = 5 + 2
print(x)
x -= 1    # x = 7 - 1
print(x)
x *= 3    # x = 6 * 3
print(x)
x /= 2    # x = 18 / 2
print(x)
```

OUTPUT

```
7
6
18
9.0
```

Using `%` and `//` to split digits. `% 10` gives the last digit; `// 10` removes it.

```
n = 357
last = n % 10    # remainder after dividing by 10
rest = n // 10   # drop the last digit
print("Last digit:", last)
print("Remaining:", rest)
```

OUTPUT

```
Last digit: 7
Remaining: 35
```

`357 ÷ 10 = 35` remainder `7`. So `357 // 10 = 35` and `357 % 10 = 7`.

REMEMBER IT

Three kinds of division: `/` = exact answer (float), `//` = quotient only ("floor"), `%` = remainder only. A number is **even** when `n % 2 == 0`.

COMMON MISTAKE

`^` is **not** power in Python — it is bitwise XOR (see bitwise operators). `2 ^ 3` gives 1, not 8. For power, always use `**`.

اردو میں سمجھیں

آپرینڈ (Operand) وہ ویلیو ہے جس پر عمل ہوتا ہے اور آپریٹر (Operator) وہ علامت ہے جو عمل کرتی ہے۔ `/` ہمیشہ اعشاریہ والا جواب دیتا ہے، `//` صرف حاصل تقسیم دیتا ہے، `%` باقی دیتا ہے اور `**` طاقت (Power) نکالتا ہے۔ `x += 2` کا مطلب ہے `x = x + 2`۔

EXAM TIP

You will be asked to evaluate expressions such as `17 // 5`, `17 % 5`, `2 ** 4` or to trace compound assignment. Show each step and write the final value with its type (e.g. `9.0` is a float, so write the `.0`).

CHECK YOURSELF

1. Predict the output:

```
x = 7
y = 2
print(x // y, x % y, x / y)
print(x ** y)
```

2. Predict the output:

```
x = 10
x += 5
x /= 4
print(x)
```

Answers: 1. `3 1 3.5` then `49`. 2. `3` (`10 + 5 = 15`; `15 // 4 = 3`).

LEARN IT

3. Relational (comparison) operators compare two values and return a **Boolean** (`True` or `False`). They are used in conditions.

Op	Description	Example (a = 10, b = 5)	Result
<code>=</code>	Equal to	<code>a == b</code>	False
<code>≠</code>	Not equal to	<code>a != b</code>	True
<code>></code>	Greater than	<code>a > b</code>	True
<code><</code>	Less than	<code>a < b</code>	False
<code>≥</code>	Greater than or equal to	<code>a ≥ 10</code>	True
<code>≤</code>	Less than or equal to	<code>a ≤ b</code>	False

```

a = 10
b = 5
print(a == b)
print(a != b)
print(a > b)
print(a < b)
print(a >= 10)
print(a <= b)

```

OUTPUT

```

False
True
True
False
True
False

```

4. Logical operators combine several conditions:

A	B	A and B	A or B	not A
False	False	False	False	True
False	True	False	True	True
True	False	False	True	False
True	True	True	True	False

- **and** — True **only if both** conditions are true.
- **or** — True if **at least one** condition is true.
- **not** — **reverses** the result (True becomes False, False becomes True).

```

a = 10
b = 5
print(a > 5 and b > 5)
print(a > 5 or b > 5)
print(not a > 5)

```

OUTPUT

```

False
True
False

```

5. Membership operators check whether a value exists inside a sequence (such as a string or list): **in** and **not in**.

```

name = "Python"
print("P" in name)
print("z" not in name)
print("p" in name)
marks = [85, 90, 75]
print(90 in marks)

```

OUTPUT

```

True
True
False
True

```

"p" in name is False because strings are case-sensitive: the word has a capital P.

WORKED EXAMPLE

Real-life conditions. A person can apply for a CNIC if they are 18 or older **and** have the documents. Marks are invalid if they are below 0 **or** above 100.

```
age = 20
has_cnic = True
print(age ≥ 18 and has_cnic)
marks = 45
print(marks < 0 or marks > 100)
```

OUTPUT

```
True
False
```

Step by step: $20 \geq 18$ is True, has_cnic is True, True and True = **True**. For marks: $45 < 0$ is False, $45 > 100$ is False, False or False = **False** (the marks are valid).

REMEMBER IT

AND is strict, OR is generous. AND is like a teacher who needs both homework and attendance; OR is like a shop that accepts cash **or** card. Order of evaluation: **not** first, then **and**, then **or**.

COMMON MISTAKE

Do not confuse = (assignment, "store") with == (comparison, "is it equal?"). Writing `if x = 5:` is a **syntax error**; the correct form is `if x == 5:`.

اردو میں سمجھیں

ریشنل آپریٹرز (Relational Operators) دو ویلیوز کا موازنہ کر کے True یا False دیتے ہیں۔ لاجیکل آپریٹرز میں **and** تب True دیتا ہے جب دونوں شرطیں درست ہوں، **or** تب جب کم از کم ایک درست ہو، اور **not** نتیجہ الٹ دیتا ہے۔ **in** اور **not in** دیکھتے ہیں کہ کوئی چیز اسٹرینگ یا لسٹ میں موجود ہے یا نہیں۔

EXAM TIP

Typical: "Evaluate $a > b$ and $b > 3$ for $a = 10$, $b = 5$ " or "Write the truth table of **and** / **or**". Write each sub-condition's value first, then combine. Remember: `=` compares, `=` assigns.

CHECK YOURSELF

1. Predict the output:

```
print(True and False or True)
print(not (5 > 3) or 2 == 2.0)
```

2. Find the error:

```
x = 5
if x = 5:
    print("five")
```

Answers: 1. True and True. (True and False = False; False or True = True. not True = False; $2 == 2.0$ is True; False or True = True.) 2. = is assignment; a comparison needs ==. Python reports SyntaxError: invalid syntax. Maybe you meant '=' or '==' instead of '='? Fix: `if x == 5:`

LEARN IT

6. Bitwise operators treat numbers as **binary digits** (bits: 0s and 1s) and work on them **bit by bit**, column by column. Think of each bit as a switch that is ON (1) or OFF (0). Python's `bin()` function shows the binary form of a number (it starts with `0b`).

- **AND (&)** — gives 1 **only if both** bits are 1, otherwise 0.
- **OR (|)** — gives 1 if **at least one** bit is 1.
- **XOR (^)** — gives 1 if the bits are **different**, 0 if they are the **same**.
- **NOT (~)** — inverts all bits (0 → 1, 1 → 0). In Python the result is the number made negative, minus 1: $\sim x = -(x + 1)$. So ~ 10 gives -11 .
- **Left shift (<<)** — moves bits left and adds 0s on the right. Each shift **multiplies by 2**.
- **Right shift (>>)** — moves bits right and removes the rightmost bits. Each shift **divides by 2** (floor division).

Bit A	Bit B	A & B	A B	A ^ B	~A
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

WORKED EXAMPLE

Worked example: a = 10, b = 6. First convert to binary using place values 8, 4, 2, 1: $10 = 8 + 2 = \mathbf{1010}$; $6 = 4 + 2 = \mathbf{0110}$. Then work column by column.

	8	4	2	1	Decimal
a = 10	1	0	1	0	10
b = 6	0	1	1	0	6
a & b	0	0	1	0	2
a b	1	1	1	0	14
a ^ b	1	1	0	0	12

- AND: only the "2" column has 1 in both → $0010 = 2$.
- OR: columns 8, 4 and 2 have at least one 1 → $1110 = 8 + 4 + 2 = \mathbf{14}$.
- XOR: columns 8 and 4 are different; column 2 is the same (both 1) → $1100 = 8 + 4 = \mathbf{12}$.
- NOT: $\sim 10 = -(10 + 1) = \mathbf{-11}$.

```
a = 10
b = 6
print(bin(a), bin(b))
print(a & b)
print(a | b)
print(a ^ b)
print(~a)
```

OUTPUT

```
0b1010 0b110
2
14
12
-11
```

Worked example: shifts. 5 in 8 bits is **0000 0101**.

Expression	Binary	Decimal	What happened
5	0000 0101	5	start
5 << 1	0000 1010	10	add one 0 on the right (5 × 2)
5 << 2	0001 0100	20	add 00 on the right (5 × 4)
20 >> 1	0000 1010	10	remove the rightmost bit (20 ÷ 2)
20 >> 2	0000 0101	5	remove two bits (20 ÷ 4)

```
a = 5          # Binary: 0000 0101
print(a << 1)   # 0000 1010
print(a << 2)   # 0001 0100
b = 20         # Binary: 0001 0100
print(b >> 1)   # 0000 1010
print(b >> 2)   # 0000 0101
print(bin(20 >> 2))
```

OUTPUT

```
10
20
10
5
0b101
```

REMEMBER IT

Shortcuts: $x \ll n = x \times 2^n$, $x \gg n = x // 2^n$, $\sim x = -x - 1$. For AND/OR/XOR always write both numbers in binary, one under the other, with the same number of bits.

COMMON MISTAKE

Do not mix up the logical and bitwise operators: `and` / `or` / `not` work on True/False conditions, while `&` / `|` / `~` work on the bits of integers. Also remember `bin(6)` prints `0b110`, without the leading zero — add zeros yourself when lining up columns.

اردو میں سمجھیں

بٹ وائز آپریٹرز (Bitwise Operators) اعداد کو بائری میں بدل کر ہر بٹ پر الگ الگ کام کرتے ہیں۔ `&` تب 1 دیتا ہے جب دونوں بٹ 1 ہوں، `|` تب جب کوئی ایک بھی 1 ہو، اور `^` تب جب دونوں بٹ مختلف ہوں۔ `~` تمام بٹ الٹ دیتا ہے، اس لیے `~10` کا جواب -11 ہے۔ لیفٹ شفٹ عدد کو 2 سے ضرب دیتا ہے اور رائٹ شفٹ 2 سے تقسیم کرتا ہے۔

EXAM TIP

Exams ask you to "evaluate `12 & 10`, `12 | 10`, `12 ^ 10`, `~5`, `7 >> 1` showing binary working". Marks are given for the binary conversion and column working, not just the final number. Always draw the column table.

CHECK YOURSELF

1. Predict the output (show the binary working):

```
print(12 & 10, 12 | 10, 12 ^ 10)
print(~5, 7 >> 1, 3 << 3)
```

2. What does `bin(13)` print?

Answers: 1. 8 14 6 and -6 3 24. 12 = 1100, 10 = 1010: AND = 1000 = 8, OR = 1110 = 14, XOR = 0110 = 6. ~5 = -(5 + 1) = -6. 7 = 111 → 11 = 3. 3 = 11 → 11000 = 24 (3 × 8). 2. 0b1101 (8 + 4 + 1).

Extra board practice

These MCQs and questions are extra to the textbook. Close the notes, attempt them, then open the key.

MCQ 1. % is:

- a. Remainder (modulo)
- b. Power
- c. AND gate
- d. Floor always in Python 3 /

MCQ 2. ** is:

- a. Exponent
- b. XOR
- c. Comment
- d. Input

MCQ 3. == tests:

- a. Equality (Boolean)
- b. Assignment
- c. Bitwise AND
- d. File open

MCQ 4. and/or/not are:

- a. Logical operators
- b. Bitwise only
- c. SQL only
- d. HTML

MCQ 5. 5 & 3 (bitwise AND) is:

- a. 1
- b. 8
- c. 15
- d. 0

MCQ 6. = vs == : = is:

- a. Assignment
- b. Comparison
- c. Power
- d. Comment

Extra short questions

1. Define operator and operand.
2. List arithmetic operators.
3. Relational operators return what type?
4. Compute $11 \% 3$ and $2**3$.

Extra long question

1. For $A=6$ (110), $B=3$ (011) show AND OR XOR bitwise, then show $6>3$ and $6==3$. Explain assignment vs comparison.

Extra answer key

MCQs: 1. A 2. A 3. A 4. A 5. A 6. A

Short

1. Operator: the symbol. Operand: the value it works on. In $3+4$, + is operator, 3 and 4 operands.
2. + - * / // % **
3. bool True/False.
4. 2 and 8.

Long

1. AND $010=2$; OR $111=7$; XOR $101=5$. $6>3$ True; $6==3$ False. = stores, == asks.